Lab Report: Kubernetes Bootcamp with Minikube

Student Name: Lu Weichi

Course: Cloud Computing

Date: 2026-05-15

Tools Used: Minikube, kubectl, Docker Desktop, WSL (Ubuntu), Git Bash

Table of Contents

Experiment 1 – Hello Minikube

Objective:

Deploy a sample application (agnhost) on a local Minikube cluster and expose it as a service accessible from the host browser.

Experimental Procedure

Start Minikube cluster

minikube start --driver=docker --image-mirror-country=cn --image-repository=registry.cn-hangzhou.aliyuncs.com/google_containers

```
lwc1116@meow:~$ minikube start
😄  minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
✨  Automatically selected the docker driver
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker driver with root privileges
❗  For an improved experience it's recommended to use Docker Engine instead of Docker Desktop.
Docker Engine installation instructions: https://docs.docker.com/engine/install/#server
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
💾  Downloading Kubernetes v1.35.1 preload ...
❗  The image 'gcr.io/k8s-minikube/storage-provisioner:v5' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/pause:3.10.1' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/coredns/coredns:v1.13.1' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/kube-scheduler:v1.35.1' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/etcd:3.6.6-0' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/kube-proxy:v1.35.1' was not found; unable to add it to cache.
❗  The image 'registry.k8s.io/kube-controller-manager:v1.35.1' was not found; unable to add it to cache.
🔥  Creating docker container (CPUs=2, Memory=3072MB) ...
❗  The image 'registry.k8s.io/kube-apiserver:v1.35.1' was not found; unable to add it to cache.
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
```

```
❌  Unable to load cached images: LoadCachedImages: stat /home/lwc1116/.minikube/cache/images/amd64/registry.k8s.io/kube
-controller-manager_v1.35.1: no such file or directory
    > kubeadm.sha256:  64 B / 64 B [--------------------------] 100.00% ? p/s 0s
    > kubectl.sha256:  64 B / 64 B [--------------------------] 100.00% ? p/s 0s
    > kubelet.sha256:  64 B / 64 B [--------------------------] 100.00% ? p/s 0s
    > kubeadm:  69.02 MiB / 69.02 MiB [--------------] 100.00% 7.06 MiB p/s 10s
    > kubectl:  55.88 MiB / 55.88 MiB [--------------] 100.00% 3.10 MiB p/s 18s
    > kubelet:  55.42 MiB / 55.42 MiB [--------------] 100.00% 1.36 MiB p/s 41s


🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Check cluster status

minikube status

```
lwc1116@meow:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

lwc1116@meow:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   108s   v1.35.1
```

Create a Deployment using agnhost image with an HTTP server on port 8080

kubectl create deployment hello-node --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080

```
lwc1116@meow:~$ kubectl create deployment hello-node \
  --image=k8s.gcr.io/agnhost:2.53 \
  -- /agnhost netexec --http-port=8080
deployment.apps/hello-node created
```

Verify Deployment and Pod

kubectl get deployment

skubectl get pods

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get deployments
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
hello-node   1/1     1            1           47m
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
hello-node-64fc5894d-fhl5b    1/1     Running   0          47m
```

View Pod logs

kubectl logs <pod-name> # replace with actual pod name

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl logs hello-node-5f76cf6ccf-br9b5
error: error from server (NotFound): pods "hello-node-5f76cf6ccf-br9b5" not found in namespace "default"
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl logs hello-node-64fc5894d-fhl5b
I0515 03:30:18.376487       1 log.go:245] Started HTTP server on port 8080
I0515 03:30:18.376794       1 log.go:245] Started UDP server on port  8081
```

Expose the Deployment as a Service (LoadBalancer type)

kubectl expose deployment hello-node --type=LoadBalancer --port=8080

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl expose deployment hello-node --type=LoadBalancer --port=8080
service/hello-node exposed
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get services
NAME         TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
hello-node   LoadBalancer   10.97.81.46    <pending>     8080:31075/TCP   9s
kubernetes   ClusterIP      10.96.0.1      <none>        443/TCP          139m
```

Access the application via Minikube tunnel

minikube service hello-node

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ minikube service hello-node
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node	8080	http://192.168.49.2:31075

```
🔗  Starting tunnel for service hello-node.
```

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node		http://127.0.0.1:46781

```
🎉  Opening service default/hello-node in default browser...
👉  http://127.0.0.1:46781
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Clean up (optional – can be done after all experiments)

kubectl delete service hello-nodekubectl delete deployment hello-node

---

Experiment 3.2 – Using kubectl to Create a Deployment

Objective:

Deploy a second application (kubernetes-bootcamp) using the same agnhost image and access it through kubectl proxy.

Experimental Procedure

Create a new Deployment

kubectl create deployment kubernetes-bootcamp --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$  kubectl delete deployment kubernetes-bootcamp
deployment.apps "kubernetes-bootcamp" deleted from default namespace
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl create deployment kubernetes-bootcamp --image=registry.k8s.io/
e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080
deployment.apps/kubernetes-bootcamp created
```

Verify Deployment and Pod

kubectl get deploymentskubectl get pods

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get deployments
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
hello-node            1/1     1            1           3h29m
kubernetes-bootcamp   1/1     1            1           12s
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
hello-node-64fc5894d-fhl5b            1/1     Running   0          3h29m
kubernetes-bootcamp-5b6dc7986c-gww5d  1/1     Running   0          18s
```

Start kubectl proxy – open a **new terminal** and keep it running

kubectl proxy

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl proxy
Starting to serve on 127.0.0.1:8001
```

Get pod name and access the app through the proxy (in the original terminal)

export POD_NAME=$(kubectl get pods -o go-template --template '{{range .items}}{{.metadata.name}}{{"\n"}}{{end}}' | grep kubernetes-bootcamp)

curl http://localhost:8001/api/v1/namespaces/default/pods/$POD_NAME:8080/proxy/

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ export POD_NAME=$(kubectl get pods -o go-template --template '{{range
.items}}{{.metadata.name}}{{"\n"}}{{end}}' | grep kubernetes-bootcamp)
curl http://localhost:8001/api/v1/namespaces/default/pods/$POD_NAME:8080/proxy/
NOW: 2026-05-15 07:11:29.076127795 +0000 UTC m=+851.014865352lwc1116@meow:~/.minikube/cache/preloaded-tarball$
```

Experiment 3.3 – Viewing Pods and Nodes

Objective:

Inspect pod details, execute commands inside a container, and understand node status.

Experimental Procedure

Describe the running pods to see detailed information

kubectl describe pods

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl describe pods
Name:             hello-node-64fc5894d-fhl5b
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Fri, 15 May 2026 11:29:34 +0800
Labels:           app=hello-node
                  pod-template-hash=64fc5894d
Annotations:      <none>
Status:           Running
IP:               10.244.0.3
IPs:
  IP:           10.244.0.3
Controlled By:  ReplicaSet/hello-node-64fc5894d
Containers:
  agnhost:
    Container ID:  docker://4895768fd9e8bcc0e4d331fed2b58a34c226b97ad69bd72d867da7b0e9f1add4
    Image:         registry.k8s.io/e2e-test-images/agnhost:2.53
    Image ID:      docker-pullable://registry.k8s.io/e2e-test-images/agnhost@sha256:99c6b4bb4a1e1df3f0b3752168c89358794d
02258ebebc26bf21c29399011a85
    Port:          <none>
    Host Port:     <none>
    Command:
      /agnhost
      netexec
      --http-port=8080
    State:          Running
      Started:      Fri, 15 May 2026 11:30:18 +0800
    Ready:          True
```

Execute a command inside the container

kubectl exec -it $POD_NAME -- /bin/sh

Inside the container:

ps auxcurl http://localhost:8080exit

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl exec -it $POD_NAME -- /bin/sh
/ # ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 /agnhost netexec --http-port=8080
   19 root      0:00 /bin/sh
   25 root      0:00 ps aux
/ # curl http://localhost:8080
NOW: 2026-05-15 07:27:22.362346184 +0000 UTC m=+1876.476597844/ # exit
lwc1116@meow:~/.minikube/cache/preloaded-tarball$
```

Experiment 3.4 – Using a Service to Expose Your App

Objective:

Create a NodePort Service, use labels to filter resources, and test external access via Minikube tunnel.

Experimental Procedure

Expose the Deployment as a NodePort Service

kubectl expose deployment kubernetes-bootcamp --type="NodePort" --port=8080

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl expose deployment kubernetes-bootcamp --type="NodePort" --port
=8080
service/kubernetes-bootcamp exposed
```

View Service details

kubectl get services

kubectl describe services/kubernetes-bootcamp

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get services
NAME                  TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
hello-node            LoadBalancer   10.97.81.46     <pending>     8080:31075/TCP   101m
kubernetes            ClusterIP      10.96.0.1       <none>        443/TCP          4h1m
kubernetes-bootcamp   NodePort       10.110.14.86    <none>        8080:32569/TCP   8s
```

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl describe services/kubernetes-bootcamp
Name:                     kubernetes-bootcamp
Namespace:                default
Labels:                   app=kubernetes-bootcamp
Annotations:              <none>
Selector:                 app=kubernetes-bootcamp
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.110.14.86
IPs:                      10.110.14.86
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  32569/TCP
Endpoints:                10.244.0.6:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Get NodePort and access via tunnel

export NODE_PORT=$(kubectl get services/kubernetes-bootcamp -o go-template='{{(index .spec.ports 0).nodePort}}')echo $NODE_PORTminikube se

rvice kubernetes-bootcamp --url

Then curl the returned URL .

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ export NODE_PORT=$(kubectl get services/kubernetes-bootcamp -o go-temp
late='{{(index .spec.ports 0).nodePort}}')
echo $NODE_PORT
32569
```

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ minikube service kubernetes-bootcamp --url
http://127.0.0.1:36051
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Label operations – use labels to query pods and add a new label

kubectl get pods -l app=kubernetes-bootcampkubectl label pods $POD_NAME version=v1kubectl describe pods $POD_NAME | grep -A 2 "Labels"

kubectl get pods -l version=v1

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get pods -l app=kubernetes-bootcampmp
kubectl label pods $POD_NAME version=v1
kubectl describe pods $POD_NAME | grep -A 2 "Labels"
kubectl get pods -l version=v1
NAME                                  READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-5b6dc7986c-gww5d   1/1     Running   0          32m
pod/kubernetes-bootcamp-5b6dc7986c-gww5d labeled
Labels:           app=kubernetes-bootcamp
                  pod-template-hash=5b6dc7986c
                  version=v1
NAME                                  READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-5b6dc7986c-gww5d   1/1     Running   0          32m
lwc1116@meow:~/.minikube/cache/preloaded-tarball$
```

Experiment 3.5 – Running Multiple Instances of Your App

Objective:

Scale the application manually, observe load balancing, and scale down.

Experimental Procedure

Scale the Deployment to 4 replicas

kubectl scale deployments/kubernetes-bootcamp --replicas=4kubectl get deployment

skubectl get pods -o wide

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl scale deployments/kubernetes-bootcamp --replicas=4
kubectl get deployments
kubectl get pods -o wide
deployment.apps/kubernetes-bootcamp scaled
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
hello-node           1/1     1            1           4h4m
kubernetes-bootcamp  1/4     4            1           35m
NAME                                  READY   STATUS              RESTARTS   AGE    IP           NODE       NOMINATED N
ODE   READINESS GATES
hello-node-64fc5894d-fhl5b            1/1     Running             0          4h4m   10.244.0.3   minikube   <none>
      <none>
kubernetes-bootcamp-5b6dc7986c-4gdgm  0/1     ContainerCreating   0          1s     <none>       minikube   <none>
      <none>
kubernetes-bootcamp-5b6dc7986c-gww5d  1/1     Running             0          35m    10.244.0.6   minikube   <none>
      <none>
kubernetes-bootcamp-5b6dc7986c-qxwjx  0/1     ContainerCreating   0          1s     <none>       minikube   <none>
      <none>
kubernetes-bootcamp-5b6dc7986c-tml2l  0/1     ContainerCreating   0          1s     <none>       minikube   <none>
      <none>
```

Test load balancing by sending multiple curl requests

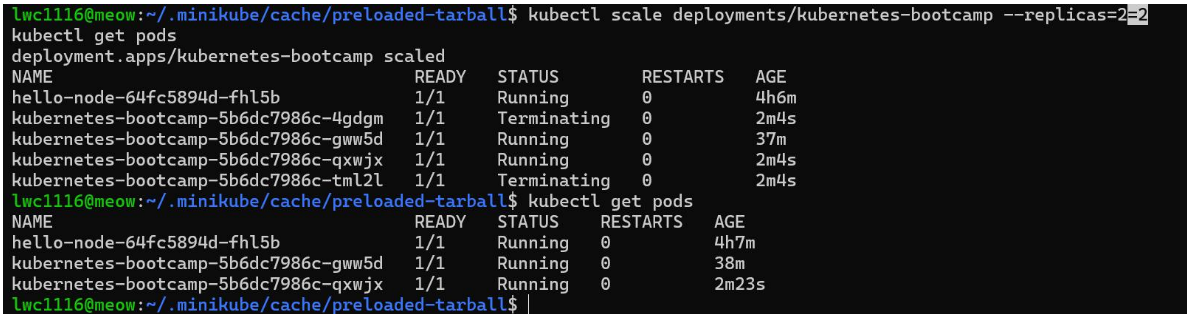
minikube service kubernetes-bootcamp --url *# get tunnel URL (keep this terminal)# In another terminal, run:*for i in {1..6}; do curl http://127.0.

0.1:xxxxx; done

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ minikube service kubernetes-bootcamp --url
http://127.0.0.1:42837
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Scale down to 2 replicas

kubectl scale deployments/kubernetes-bootcamp --replicas=2kubectl get pods



```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl scale deployments/kubernetes-bootcamp --replicas=2=2
kubectl get pods
deployment.apps/kubernetes-bootcamp scaled
NAME                                   READY   STATUS        RESTARTS   AGE
hello-node-64fc5894d-fhl5b             1/1     Running       0          4h6m
kubernetes-bootcamp-5b6dc7986c-4gdgm   1/1     Terminating   0          2m4s
kubernetes-bootcamp-5b6dc7986c-gww5d   1/1     Running       0          37m
kubernetes-bootcamp-5b6dc7986c-qxwjx   1/1     Running       0          2m4s
kubernetes-bootcamp-5b6dc7986c-tml2l   1/1     Terminating   0          2m4s
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
hello-node-64fc5894d-fhl5b             1/1     Running   0          4h7m
kubernetes-bootcamp-5b6dc7986c-gww5d   1/1     Running   0          38m
kubernetes-bootcamp-5b6dc7986c-qxwjx   1/1     Running   0          2m23s
lwc1116@meow:~/.minikube/cache/preloaded-tarball$
```

Experiment 3.6 – Performing a Rolling Update and Rollback

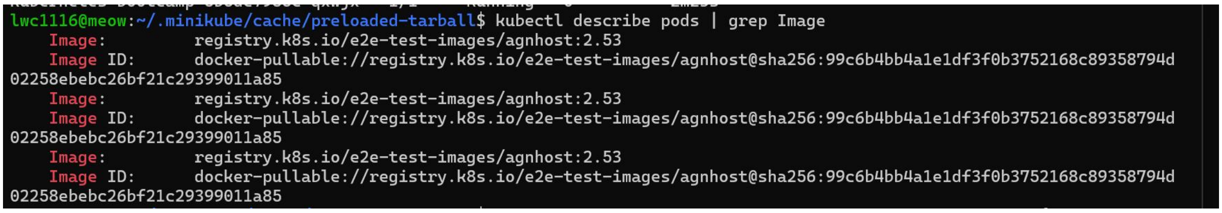
Objective:

Update the container image to a new version, simulate a failed update (image pull error), and roll back to the previous stable version.

Experimental Procedure

Check the current image version

```
kubectl describe pods | grep Image
```



```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl describe pods | grep Image
    Image:          registry.k8s.io/e2e-test-images/agnhost:2.53
    Image ID:       docker-pullable://registry.k8s.io/e2e-test-images/agnhost@sha256:99c6b4bb4a1e1df3f0b3752168c89358794d
02258ebebc26bf21c29399011a85
    Image:          registry.k8s.io/e2e-test-images/agnhost:2.53
    Image ID:       docker-pullable://registry.k8s.io/e2e-test-images/agnhost@sha256:99c6b4bb4a1e1df3f0b3752168c89358794d
02258ebebc26bf21c29399011a85
    Image:          registry.k8s.io/e2e-test-images/agnhost:2.53
    Image ID:       docker-pullable://registry.k8s.io/e2e-test-images/agnhost@sha256:99c6b4bb4a1e1df3f0b3752168c89358794d
02258ebebc26bf21c29399011a85
```

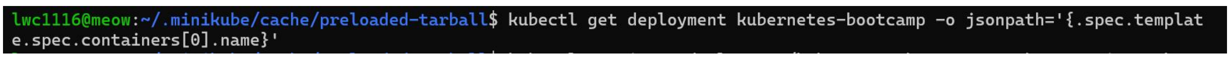
Attempt to update to version 2.52 (this may fail due to network or missing tag)

First get the container name:

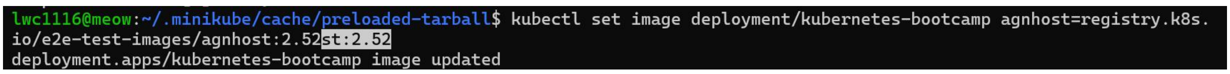
```
kubectl get deployment kubernetes-bootcamp -o jsonpath='{.spec.template.spec.containers[0].name}'
```

Then update:

```
kubectl set image deployment/kubernetes-bootcamp agnhost=registry.k8s.io/e2e-test-images/agnhost:2.52
```



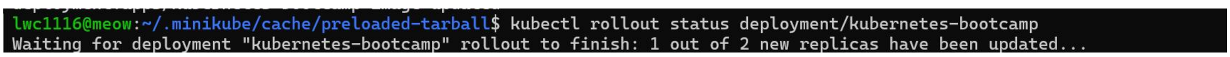
```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl get deployment kubernetes-bootcamp -o jsonpath='{.spec.templat
e.spec.containers[0].name}'
```



```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl set image deployment/kubernetes-bootcamp agnhost=registry.k8s.
io/e2e-test-images/agnhost:2.52st:2.52
deployment.apps/kubernetes-bootcamp image updated
```

Monitor the rollout status

```
kubectl rollout status deployment/kubernetes-bootcamp
```



```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl rollout status deployment/kubernetes-bootcamp
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 out of 2 new replicas have been updated...
```

Check the failed pods

```
kubectl get pods
```

```
PS C:\Users\21315> wsl
lwc1116@meow:/mnt/c/Users/21315$ kubectl get pods -w
NAME                                  READY   STATUS             RESTARTS   AGE
hello-node-64fc5894d-fhl5b            1/1     Running            0          4h12m
kubernetes-bootcamp-5b6dc7986c-gww5d  1/1     Running            0          43m
kubernetes-bootcamp-5b6dc7986c-qxwjx  1/1     Running            0          8m4s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ImagePullBackOff   0          2m15s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ErrImagePull       0          2m20s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ImagePullBackOff   0          2m33s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     Terminating        0          3m25s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     Terminating        0          3m25s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     Terminating        0          3m26s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ContainerStatusUnknown   0          3m26s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ContainerStatusUnknown   0          3m26s
kubernetes-bootcamp-75f859864d-lmbsc  0/1     ContainerStatusUnknown   0          3m26s
```

Roll back to the previous stable version

kubectl rollout undo deployment/kubernetes-bootcamp

```
^Clwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl rollout undo deployment/kubernetes-bootcamp
deployment.apps/kubernetes-bootcamp rolled back
```

Verify the rollback succeeded

kubectl rollout status deployment/kubernetes-bootcampkubectl get podskubectl describe pods | grep Image

```
lwc1116@meow:~/.minikube/cache/preloaded-tarball$ kubectl rollout status deployment/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out
lwc1116@meow:~/.minikube/cache/preloaded-tarball$
```

Final Cleanup (after all experiments)

bash

kubectl delete deployment kubernetes-bootcampkubectl delete service kubernetes-bootcampkubectl delete deployment hello-nodekubectl delete service hello-nodeminikube stop

---

Conclusion

In this lab, I successfully completed all six experiments using Minikube and kubectl. Key learnings include:

How to start a local Kubernetes cluster with Minikube (using Docker driver and a domestic image mirror for reliability).
How to create and manage Deployments, view Pods, and inspect logs.
How to expose applications inside the cluster to the outside world using Services (NodePort and LoadBalancer types) and the `minikube service` tunnel.
How to use labels to organise and filter Kubernetes resources.
How to scale applications up and down, and verify load balancing across multiple Pod replicas.How to perform rolling updates, handle failed image pulls, and roll back to a previous version.

The experiments demonstrated Kubernetes' self-healing, service discovery, and declarative update capabilities. Although the original `kubernetes-bootcamp` image was unavailable, the `agnhost` image served as a perfect alternative, validating all core concepts.